Développement Web côté client

KX07KAR7

Exercice 1 : Start Bootstrap

Modifier le fichier `index.html` dans le dossier `projetweb` .

- Placer la balise meta dans la section `head` de la page : `<meta name="viewport" content="width=device-width, initial-scale=1">`
- Inclure bootstrap CSS, placer la balise `link` dans la section `head` de la page : `<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-T3c6CoIi6uLrA9TneNEoa7RxnatzjcDSCmG1MXxSR1GAsXEV/Dwwykc2MPK8M2HN" crossorigin="anonymous">`
- Inclure bootstrap JS, placer la balise `script` juste avant la balise fermante `/body` : `<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js" integrity="sha384-C6RzsynM9kWDrMNeT87bh95OGNyZPhcTNXj1NW7RuBCsyN/o0jlpcV8Qyq46cDfL" crossorigin="anonymous"></script>`
- Pour résumé, les balises doivent être placés comme dans l'example ci-dessous :

```
   <!doctype html>
   <html lang="en">
     <head>
       <meta charset="utf-8">
       <meta name="viewport" content="width=device-width, initial-scale=1">
       <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet"
integrity="sha384-T3c6CoIi6uLrA9TneNEoa7RxnatzjcDSCmG1MXxSR1GAsXEV/Dwwykc2MPK8M2HN" crossorigin="anonymous">
     </head>
     <body>
       ...
       <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-C6RzsynM9kWDrMNeT87bh95OGNyZPhcTNXj1NW7RuBCsyN/o0jlpcV8Qyq46cDfL" crossorigin="anonymous">
</script>
     </body>
   </html>
```

- Ouvrir la page `index.html` dans un navigateur, appliquer la class `table` au tableau dans cette page.

```
<table id="data" class="table">
```

Exercice 2 : Interface "Apprendre"

L'objectif de cet exercice est de construire une interface "apprendre" avec les composants Bootstrap, en notamment les tableaux, les boutons et les composant d'un formulaire. Dans le dossier `projetweb` , créer le fichier `apprendre.html`

- Dans cette interface, faire afficher les questions lors du chargement de la page.
  - Copier le code HTML du fichier `index.html` à la page `apprendre.html`
  - Ajouter l'évènement `onload` à la balise `body` avec la valeur `loadXMLDoc()`
  - Vérifier que les données soient bien affichées lors du chargement de la page
- Créer des composants permettant la filtrage de questions : par thématique (et plus tard : par niveau, par points associés), ...
  - Ajouter un dropdown box permettant de sélection la thématique de questions et un bouton pour appliquer le filtrage :

    Tout Filtrer

    ```
    <select id="thematique" class="form-select">
        <option value="Tout">Tout</option>
        <option value="Terre">Terre</option>
        <option value="Eau">Eau</option>
        <option value="Air">Air</option>
    </select>
    <input type="button" onclick="filterDataTable()" value="Filtrer" class="btn-primary"/>
    ```

  - Code JavaScript pour récupérer le choix de l'utilisateur :

```
let slThematique = document.getElementById("thematique");
let theme = slThematique.options[slThematique.selectedIndex].value;
alert(theme);
```

- Mettre à jour le ficher `bdd.xml` pour que chaque question a une thématique associée.
- Déployer la nouvelle version de votre BDD sur le serveur obiwan. Pour faciliter le processus, on utilisera la commande `scp chemin_vers_votre_bdd_xml nom_utilisateur@obiwan.univ-brest.fr:public_html/data`
- Mettre à jour le code de la fonction `displayDataTable()` pour afficher la nouvelle BDD (avec l'attribut thématique)
  - Afin d'assurer que la nouvelle BDD soit téléchargée par le navigateur, **effacer les données de navigation de votre navigateur** et ensuite recharger la page `apprendre.html`.
  - **ATTENTION** : Pour tous les problèmes liés à l'affichage des données, avant poser des questions, vérifier :
    - Les données XML sont valides
    - Le fichier bdd.xml est bien déployé sur le serveur
    - Les données de navigation sont bien effacées
    - Effectuer une première recherche d'erreur avec l'outil de développeur (F12)
- Analyzer la fonction `displayDataTable()` et noter la explication pour chaque ligne de code dans le fichier README.txt
- Mettre en oeuvre de la fonction `filterDataTable()`. Utiliser le code de la fonction `displayDataTable()` comme un point de départ.
  - Note : en arrivant à cette étape, vous avez 4 fonctions dans le fichier `scripts.js` : loadXMLDoc(), loadData(), displayDataTable(), filterDataTable()

- Mettre à jour le fichier `README.txt` avec un manuel d'utilisation de l'interface "apprendre".
- À faire après la séance de TP1 :
  - Mettre en ouvre la pagination.
  - Créer la page detail.html pour afficher le détail d'une question

Exercice 3 : Interface "Index" et Interface "Jeu"

La page `index.html` sera utilisée comme le point d'entrée principal.

- L'utilisateur peut choisir un **lot** de questions pour commencer une partie du jeu Chaque partie contient 10 questions.
  - Afficher les ensembles de questions en utilisant des divisions
  - Pour cela, proposer une mise à jour de la base de données
- L'utilisateur peut également choisir de commencer une partie sur une **thématique**.

Après avoir fait son choix, l'utilisateur devra être réorienté vers la page `jeu.html`. Envoyer l'information concernant le choix d'utilisateur à cette page dans une chaîne de requête. (On ne gère pas l'implantation du jeu pour l'instant).

Exercice 4 : Interface "Cartes"

Dans cet exercice, l'objectif est de créer une base de données de cartes Pokémon et de les afficher dans une page web.

- Créer un fichier XML dans le répertoire `data` : `bdd_cartes.xml`
- Créer un fichier HTML dans le répertoire `projetweb` : `cartes.html`
- Analyser des exemples des cartes Pokémon (trouvés sur les sites par exemple limitlesstcg), puis identifiez les attributes à enregistrer dans la base de données. Identifier au moins 5 informations utiles permettant de décrire une carte.
- Mettre à jour la base de données `bdd_cartes.xml`. Elle devra comporter :
  - Un élément racine nommé `cartes`.
  - Un élément `carte` pour chaque carte
- Déployer la base de données sur serveur `obiwan` (dans le répertoire `public_html/data`)
- Mettre à jour le fichier `cartes.html` afin d'afficher les cartes
  - L'affichage devra être aussi proche que possible de celui d'une carte réelle
  - À faire après la séance de TP1 : Implanter des fonctions de pagination et de filtrage

Exercice 5 (facultatif) : Interface "Login"

Dans cet exercice, l'objectif est de créer une BDD d'utilisateurs avec leurs logins/mdps. En suite, il faut créer une page d'authentification. Plus tard, on demandera les utilisateurs de leurs connecter avant accéder à l'interface "Apprendre" (avec l'usage de Cookie/Web Storage)

- Dans dossier `projetweb` , créer fichier `login.xml`. Dans ce fichier, créer un formulaire de authentification
- Remplir le fichier XML avec 2-3 utilisateurs. Pour chaque utilisateur, on a besoin d'un ID et un mot de passe.
- Les MDPs ne sont pas sauvegardé en texte claire. Nous utiliserons une fonction de hachage (hash function) pour calculer le hash d'un MDP. Ensuite, le hash sera mis dans la BDD. Vous pouvez utiliser des outils en lignes pour faire le calcul : https://www.sha1.fr/

- Lorsque l'utilisateur se connecte, calculer le hash SHA1 du mot de passe et le hash dans la base de donnée. On utilisera la bibliothèque CryptoJS pour effectuer le calcul en JavaScript
  - Pour inclure CryptoJS : `<script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/4.1.1/crypto-js.min.js" integrity="sha512-E8QSvWZ0eCLGk4km3hxSsNmGWbLtSCSUcewDQPQWZF6pEU8GlT8a5fF32wOl1i8ftdMhssTrF/OhyGWwonTcXA==" crossorigin="anonymous" referrerpolicy="no-referrer"></script>`
  - Documentation : https://cryptojs.gitbook.io/docs/
- Réorienter l'utilisateur vers la page `apprendre.html` lors d'une authentification réussite.